

MACHINE LEARNING IN FINANCE

MODULE 5 | LESSON 1

SUPPORT VECTOR MACHINES

Reading Time	20 minutes
Prior Knowledge	Machine Learning classification, linear and non-linear data, bias and variance
Keywords	Support Vector, Margin, Kernel

In the previous module, we covered two types of classification methods, namely the Naïve Bayes and Decision Trees approach. In this lesson, we will cover another popular method called Support Vector Machines.

1. Introduction

Support vector machines is an approach to solving classification problems and has grown in popularity since its introduction in the 1990s. Like logistic regression, it is effective for linearly separable data, or data that can be separated by a linear boundary. However, there are hyperparameters that can be used to adjust the boundary to deal with non-linear data. This boundary is what we call the separating hyperplane.

2. Separating Hyperplane

A hyperplane is simply a flat linear plane for any p -dimensional space. An example would be the straight line in a 2-dimensional space as can be seen in fig. 1, i.e., for $p=2$ dimensions, the hyperplane is a flat $p-1=1$ dimensional subspace. In figure 1 below, the blue line is a hyperplane in a two-dimensional space.

Figure 1: A Linear Boundary Separating a Binary Target Class Data

Mathematically, in this example, the hyperplane can be represented by the equation of a straight line,
$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0, \tag{1}$$
 hence, in general for p -dimensions,

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = 0, \tag{2}$$
 or,

$$\sum_{j=1}^p \beta_j X_j + \beta_0 = 0.$$

Suppose the equation of the separating hyperplane represented in fig. 1 by the blue line is given by,

$$-4 + 4X_1 + X_2 = 0.$$

Now if we had an observation with (X_1, X_2) coordinates of $(6, -10)$, then substituting this point into (4) would result in,

$$-4 + 4(6) + (-10) = 10 > 0.$$

This observation would be above the hyperplane, along with class 1 (red triangles). The rule for classifying target classes for any observation of (X_1, X_2) would be,

$$-4 + 4X_1 + X_2 > 0 \text{ Then Class 1,}$$

$$-4 + 4X_1 + X_2 < 0 \text{ Then Class 0.}$$

The question you may ask is what do we do with an observation that is on the boundary, i.e.,

$$-4 + 4X_1 + X_2 = 0 \text{ ?}$$

To address this, we will need to look at how this hyperplane is derived, and to do this, we look at the Maximal Margin Classifier.

3. Maximal Margin Classifier

You will want to refer to fig. 2 to aid in your understanding of this section. In the example shown in fig. 2, the optimal classifier for two predictors is shown by the blue line. This optimal hyperplane is constructed such that there are equidistant "buffers" on either side shown by the dashed lines. The perpendicular distance between the dashed lines is known as the margin and the optimal hyperplane is achieved when this margin is maximized. Another way of thinking about this is the maximal margin classifier maximizes the distance to the nearest data point. You will notice that the dashed lines touch the data on either side, but it is only necessary to touch the data points on one side before the margin is achieved. These buffers shown by the dashed lines are known as support vectors.

Figure 2: Optimal Boundary Separating Two Classes

For this binary classification problem for predictors X_1 and X_2 , the optimized boundary is obtained by maximizing the margin, M , subject to:

$$\sum_{j=1}^2 \beta_j^2 = 1.$$

For a higher dimension of predictors say p , this is extended to the constraint,

$$\sum_{j=1}^p \beta_j^2 = 1. \tag{4}$$

The constraint in equation (4) is to ensure we look at the perpendicular distance from the j th data point to the hyperplane.

Instead of class 0 and 1 for our binary problem, let us suppose the class labels are -1 and 1 instead, i.e., y_i is -1 or 1 for the i th observation. From (3), a correct classification would be

$$y_i(\sum_{j=1}^p \beta_j X_{ij} + \beta_0) > 0, \tag{5}$$

and hence misclassifications as

$$y_i(\sum_{j=1}^p \beta_j X_{ij} + \beta_0) \leq 0. \tag{6}$$

A method, called the Perceptron algorithm, determines the weights, β_j , by looking at the misclassifications in (6) and updating the weights in an iterative way. The details of this algorithm will not be covered in this course.

For an extra safeguard, think of the perpendicular distance of the hyperplane to the support vector as D . For this extra safeguard, we can change (5) to be

$$y_i(\sum_{j=1}^p \beta_j X_{ij} + \beta_0) > D. \tag{7}$$

One thing to note is that SVMs do not output probabilities of the target class; however, some methods do exist, such as Platt scaling and isotonic regression, to obtain this. Thus far, we have only considered linearly separable data, but often in practice, this may not be the case. The perceptron algorithm does not perform well for non-linearly separable data. This brings us to Hard and Soft margins.

4. Types of Margins

Hard margins are when no misclassifications occur or are allowed. Soft margins are when the number of misclassifications can be specified. A hyperparameter called the regularization parameter, c , specifies the importance of having misclassifications. The higher c , the greater the penalty, i.e. it is very important to have no misclassifications. When c is increased we move towards a hard margin, i.e., no misclassifications. The problem with using a hyperplane that perfectly separates classes in the training dataset is that it may not achieve the same performance for unseen or test data. This is the problem of over-fitting. When c is increased, we are actually taking a more conservative approach by using a very small margin, i.e., the dashed lines in fig. 2 are much closer to the hyperplane (solid blue line). A smaller c pushes these margins further away since we are allowing the classifier to accommodate more misclassifications. In other words, a small c enables a soft margin, at the cost of some misclassifications. Slack variables are measures of the distance of the misclassified points from the margins, i.e. those points that don't sit perfectly on the correct side of the margin. The parameter

C is the constraint on slack variables. It is important to note that some books use C inversely by defining it as the upper bound of the sum of all slack variables. We will use the notation C to keep in line with how it is used in python. Refer to <https://stats.stackexchange.com/q/488107> for the mathematical interpretation. New data introduced into the training set can change the hyperplane; however, it is important to note that only points on the margin and on the wrong side of the hyperplane would affect a change. This quality emphasizes the insensitivity to outliers. The gamma hyperparameter can be adjusted such that nearby or far away points to the decision boundary have an influence. Low gamma takes far away points and vice versa. We finally get to the problem of non-linear data and the approach of SVM to overcome this via kernels.

5. Kernels

Let's watch the video below to get an idea about what a Kernel does.

```
In [1]: from IPython.display import VimeoVideo  
  
VimeoVideo("781662374", h="d05a258e6d", width=600)
```

Out[1]:



[Access Video Transcript here](#)

Suppose the target classes are distributed in the feature space between two predictors as shown in the left plot of fig. 3.

Figure 3: Left: Non-linearly separable data. Right: Data projected into an extra dimension Z such that $Z = X_1^2 + X_2^2$.

Using the hard margin approach, a straight line would not split the classes appropriately. An approach may be to add an extra predictor that is a function of the original such that

$$Z = X_1^2 + X_2^2$$

Projecting the feature space into an extra dimension would show the distribution of the target classes in the right plot of fig. 3. We now see that a plane can separate the target classes efficiently. This mapping into higher dimensions is done by kernels. SVM algorithms have a number of kernels to choose from, such as Polynomial Kernel and Gaussian Radial Basis Function (RBF) to name a few. For a more comprehensive list, please refer to <https://data-flair.training/blogs/svm-kernel-functions/>. One thing to note is that this illustrative example was merely two predictors originally; therefore, with more predictors the optimal transformation is not only difficult find but also to compute. Kernels achieve this in a more memory efficient way. The detailed mathematics behind kernels is not part of the scope of this module, but the interested reader is referred to <https://www.analyticsvidhya.com/blog/2020/10/the-mathematics-behind-svm/>. What we need to understand is that the solution to calculating the optimal hyperplane depends on the dot or inner product of the observations. The inner product for two vectors $a = (a_1, a_2, \dots, a_n)$ and $b = (b_1, b_2, \dots, b_n)$ of equal length n is

$$\langle a, b \rangle = \sum_{i=1}^n a_i b_i \tag{7}$$

From (7), you may notice that this is 'somewhat' how we obtained the calculation of Z for the example above, i.e.,

$$Z = \sum_{j=1}^2 \langle X_{j,i}, X_{j,i'} \rangle = K(X_{j,i}, X_{j,i'})$$

for two different observations i and i' . We use K to denote the kernel.

In general, a linear support vector can be represented as

$$y(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle X, X_i \rangle$$

for some new point X , where the parameters α_i are estimates over each observation i . This implies that inner products between the new point X and all other observations need to be calculated to estimate α ; however, α is zero for all observations that are not support vectors. We therefore see that determining the α s is made less computationally intensive.

The Gaussian RBF kernel has the below formula instead,

$$K(X_i, X_{i'}) = \exp(-\gamma \|X_i - X_{i'}\|^2)$$

where $\|X_i - X_{i'}\|$ is the euclidean distance between observation X_i and $X_{i'}$. This may look familiar to you if you've seen the formula for a Gaussian equation $\exp(-\gamma \|X - \mu\|^2)$ for mean μ . The parameter γ is synonymous with the width of the Gaussian distribution or standard deviation, σ , i.e., $\gamma = 1/2\sigma^2$. Hence, this is a hyper parameter provided by the user and a too large γ can lead to over-fitting. The Gaussian Kernel is often used when the original data is non-linearly separable and is sometimes the default kernel

in programming languages. It is best to know the default kernel used in the SVM model and experiment with different kernels.

6. Conclusion

In this lesson, we covered the mathematics and methodology behind support vector machines. We looked at the hyperplane and support vectors that form the decision boundaries for separating a binary target class. Methods to possibly overcome non-linearly separable data such as kernels were examined. In the next lesson, we will put SVM into practice with a problem in the finance field.

References

DataFlair. "Kernel Functions-Introduction to SVM Kernel and Example." <https://data-flair.training/blogs/svm-kernel-functions/>

James, Gareth, et al. *An Introduction to Statistical Learning: With Applications in R*. 2nd ed., Springer, 2021.

Radhika. "The Mathematics Behind Support Vector Machine Algorithm (SVM)." *Analytic Vidhya*, 23 Oct. 2020. <https://www.analyticsvidhya.com/blog/2020/10/the-mathematics-behind-svm/>

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.